

CS 103000
Prof. Madeline Blount

Week 10



Dall-E 2: cats learning C++ in the forest on '90's technology



housekeeping:

- welcome back!
- midterm hackathon + grades soon; extra credit soon
- final project release in next few weeks
- a little review of functions + files

HACKATHON!

solid projects:

- brute force decryption of every key
- at least some attempt, partial credit for the program itself to surface which key is correct
- clean implementation of an LCG with unique parameters that give pseudorandom numbers with a period that doesn't repeat too soon
- team members: basic understanding of each section of the code

strong projects:

- devised a method to test decrypted text as english (counting common letters, testing common words, most robust would be full frequency distribution or Index of Coincidence), after brute force each key
- there is in fact a method for finding the keyword without a list, Kasiski test, estimate key length first
- creative, reversible encryption that prints out a working key
- clean code with clear comments in backend, creative or seamless front-end
- team members: well-explained methods, understanding of randomness

losing points:

- lack of citation !!!
- errors in decryption, typos, debug/proofread
- encryption not reversible
- missed opportunity for vectors or loops
- individual form missing explanation for sections



FILE STRUCTURE



main.cpp

#include



Function
declarations

myheader.h



Function
definitions

myfunctions.cpp

```
compile: g++ main.cpp myfunctions.cpp
```



CLASSES + OBJECTS = NEW PARADIGM

- Procedural programming
- Functional programming
- Object-oriented programming (OOP)
 - *More* abstraction
 - *More* control





OBJECT ORIENTED = NEW PARADIGM, classes

- Bjarne Stroustrup made C++ with classes in mind
- CLASS = make your own data type
- WHEN make a class?
 - When some data type will repeat
 - When your data will have multiple attributes
 - “Model” something in the world



Lightbulb

isOn (boolean)

watts (number)

turnOn() (func)



Microphone

color (string)

isAnalog (boolean)

record() (func)



Animal

color (string)

age (number)

speak() (func)



Date

month (number)

year (number)

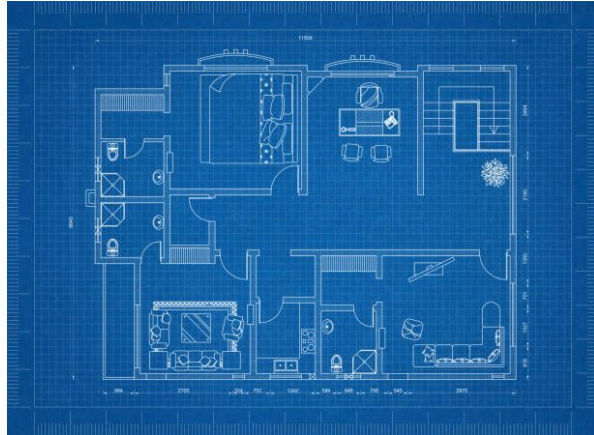
toString() (func)

CLASS vs. OBJECT

- Class is the blueprint; object is an INSTANCE of the class

CLASS

blueprint



OBJECT

building



CLASS vs. OBJECT

- Class is the blueprint; object is an INSTANCE of the class

CLASS

cookie cutter



OBJECT

cookie

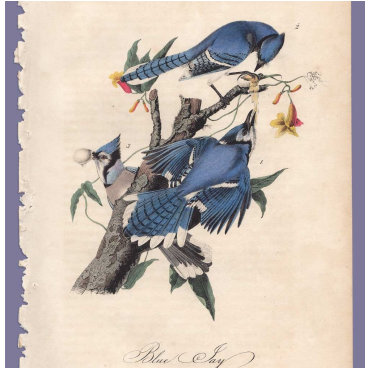


CLASS vs. OBJECT

- Class is the blueprint; object is an INSTANCE of the class

CLASS

species



OBJECT

Actual birds in the world



MAKING A CLASS

- Class definition
- Attributes, or member data (what are the THINGS that define the objects in this class? Size, name, color, rate, cost, etc.)
- Methods, or member functions (what are the things you can DO with the objects in this class)



FILE STRUCTURE

imagine class called Movie...



main.cpp

#include



Movie.h

Class encapsulation,
(attributes, member
data, declarations)

#include



Movie.cpp

Class methods
(function
definitions)

```
compile: g++ main.cpp Movie.cpp
```



CONSTRUCTOR

<https://cs103-3proton.glitch.me/>

- object initialization function, runs automatically
- declared in **public** encapsulation
- Funny definitions syntax ... **DOES NOT HAVE RETURN TYPE!**

example: class named Neighbor

```
Neighbor::Neighbor() {  
  
}
```